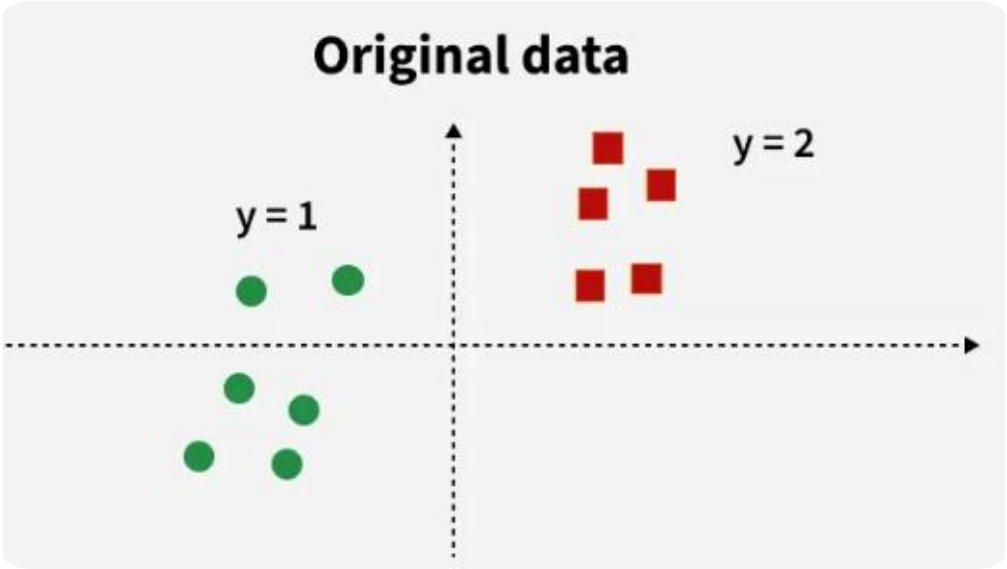
Naïve Bayes



- Naive Bayes is a probabilistic machine learning algorithm used for classification tasks. It predicts the class of a data point based on probability calculations while assuming that all features are independent. Despite this simplifying assumption, Naive Bayes is highly effective in real-world applications such as spam detection, document classification, and sentiment analysis.
- The Naive Bayes classifier is a popular supervised machine learning algorithm used for classification tasks such as **text classification**. It belongs to the family of generative learning algorithms, which means that it models the distribution of inputs for a given class or category. This approach is based on the assumption that the features of the input data are conditionally independent given the class, allowing the algorithm to make predictions quickly and accurately.

Example of Naïve Bayes Algorithm

- For example, if a fruit is red, round, and about 3 inches wide, we might call it an apple. Even if these things are related, each one helps us decide it's probably an apple. That's why it's called 'Naive.
- An NB model is easy to build and particularly useful for very large data sets. Along with simplicity, Naive Bayes is known to outperform even highly sophisticated classification methods.
- Bayes theorem provides a way of computing posterior probability P(c|x) from P(c), P(x) and P(x|c). Look at the equation below:

Likelihood

Class Prior Probability

$$P(c \mid x) = \frac{P(x \mid c)P(c)}{P(x)}$$

Posterior Probability

Predictor Prior Probability

$$P(c \mid \mathbf{X}) = P(x_1 \mid c) \times P(x_2 \mid c) \times \cdots \times P(x_n \mid c) \times P(c)$$

- $P(c|x)$ is the posterior probability of *class* (c, *target*) given *predictor* (x, *attributes*).
- $P(c)$ is the prior probability of *class*.
- $P(x|c)$ is the likelihood which is the probability of the *predictor* given *class*.
- $P(x)$ is the prior probability of the *predictor*.

Sample

- It is called “Naive” because it assumes that the presence of one feature does not influence the others, and “Bayes” because it is based on Bayes’ Theorem.
- Consider a fictional dataset that represents weather conditions for playing a game of golf. Based on these conditions, each record is classified as either suitable (“Yes”) or unsuitable (“No”) for playing golf. The dataset can be represented in tabular form as follows.

	Outlook	Temperature	Humidity	Windy	Play Golf
0	Rainy	Hot	High	False	Yes
1	Rainy	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Sunny	Mild	High	False	No
4	Sunny	Cool	Normal	False	Yes
5	Sunny	Cool	Normal	True	No
6	Overcast	Cool	Normal	True	Yes
7	Rainy	Mild	High	False	No
8	Rainy	Cool	Normal	False	Yes
9	Sunny	Mild	Normal	False	Yes
10	Rainy	Mild	Normal	True	Yes
11	Overcast	Mild	High	True	Yes
12	Overcast	Hot	Normal	False	Yes
13	Sunny	Mild	High	True	No

How it Works?

Thus, Bayes' theorem becomes:

$$P(y|x_1, ..., x_n) = \frac{P(y) \cdot \prod_{i=1}^{n} P(x_i|y)}{P(X)}$$

Since the denominator is constant for a given input, we can write:

$$P(y|x_1, ..., x_n) \propto P(y) \cdot \prod_{i=1}^{n} P(x_i|y)$$

- Naive Bayes: How It Works
- 1. Terminology
- Consider a classification problem, such as predicting whether a person will play golf based on weather conditions.
- **y** represents the class label (for example, “Yes” or “No” for playing golf).
- **X = (x_1, x_2, ..., x_n)** represents the feature vector (such as Outlook, Temperature, Humidity, and Wind).
- An example data point from the dataset could be:
- **X = (Rainy, Hot, High, False), y = No**
- This corresponds to the question:
- *What is the probability that a person will not play golf given that the weather is rainy, hot, with high humidity and no wind?*
- 2. The Naive Assumption
- The term *“naive”* in Naive Bayes comes from the assumption that all features are conditionally independent given the class label. In other words, the occurrence of one feature does not affect the others when the class is known. Mathematically, this is expressed as:

Let's take a dataset used for predicting if golf is played based on:

- **Outlook**: Sunny, Rainy, Overcast
- **Temperature**: Hot, Mild, Cool
- **Humidity**: High, Normal
- **Wind**: True, False

Play		P(Yes)/P(No)
Yes	9	9/14
No	5	5/14
		%

Outlook

	Yes	No	P(yes)	P(no)
Sunny	3	2	3/10	2/4
Overcast	4	0	4/10	0/4
Rainy	3	2	3/10	2/4
Total	10	4	100%	100%

Humidity

	Yes	No	P(yes)	P(no)
Hot	3	4	3/9	4/5
Normal	6	1	6/9	1/5
Total	9	5	100%	100%

Temperature

	Yes	No	P(yes)	P(no)
Hot	2	2	2/9	2/5
Mild	4	2	4/9	2/5
Cool	3	1	3/9	1/5
Total	9	5	100%	100%

Wind

	Yes	No	P(yes)	P(no)
False	6	2	6/9	2/5
True	3	3	3/9	3/5
Total	9	5	100%	100%

Calculation

- **Example Input:**
- X = (Sunny, Hot, Normal, False)
- **Objective:**
- Predict whether golf will be played (**Yes** or **No**).
- 5. Pre-computation from the Dataset
- **Class Probabilities:**
- From a dataset containing 14 instances:
- P(Yes) = 9/14
- P(No) = 5/14
- These probabilities represent the prior likelihood of playing or not playing golf before considering any specific weather conditions.

Feature	Value	P (Value | Yes)	P (Value | No)
Outlook	Sunny	2/9	3/5
Temperature	Hot	2/9	2/5
Humidity	Normal	6/9	1/5
Wind	False	6/9	2/5

Calculate Posterior Probabilities

For Class = Yes:

$$P(\text{Yes} \mid \text{today}) \propto \frac{2}{9} \cdot \frac{2}{9} \cdot \frac{6}{9} \cdot \frac{6}{9} \cdot \frac{9}{14}$$

$$P(\text{Yes} \mid \text{today}) \approx 0.02116$$

For Class = No:

$$P(\text{No} \mid \text{today}) \propto \frac{3}{5} \cdot \frac{2}{5} \cdot \frac{1}{5} \cdot \frac{2}{5} \cdot \frac{5}{14}$$

$$P(\text{No} \mid \text{today}) \approx 0.0068$$

7. Normalize Probabilities

To compare:

$$P(\text{Yes} \mid \text{today}) = \frac{0.02116}{0.02116+0.0068} \approx 0.756$$

$$P(\text{No} \mid \text{today}) = \frac{0.0068}{0.02116+0.0068} \approx 0.244$$

Feature	Value	P (Value | Yes)	P (Value | No)
Outlook	Sunny	2/9	3/5
Temperature	Hot	2/9	2/5
Humidity	Normal	6/9	1/5
Wind	False	6/9	2/5

$$P(\text{Yes} \mid \text{today}) > P(\text{No} \mid \text{today})$$

The model predicts: ***Yes (Play Golf)***

Naïve Bayes for Continuous Features

For continuous features, we assume a Gaussian distribution:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i-\mu_y)^2}{2\sigma_y^2}\right)$$

Where:

- μ_y is the mean of feature x_i for class y
- σ_y^2 is the variance of feature x_i for class y

Addressing Numerical Underflow in Naive Bayes

- Numerical underflow occurs when values become so small that they can no longer be accurately represented using a computer's floating-point precision and are approximated as zero. This problem can arise in Naive Bayes classification because the algorithm multiplies many small probabilities together, causing the resulting value to shrink rapidly and potentially underflow.
- From a numerical stability perspective, the main challenge with Naive Bayes is that it computes predictions by multiplying probabilities together. Since probabilities are values between 0 and 1, multiplying many small probabilities can quickly produce numbers that are extremely close to zero.
- When a model multiplies several very small likelihoods for a given class (for example, (1×10^{-10})), the resulting product can become even smaller, such as (1×10^{-100}). Such values may fall below the machine's numerical precision and be rounded to zero. When this happens, the classifier can no longer distinguish between probabilities that have underflowed to zero, leading to inaccurate predictions.
- This issue can significantly degrade the performance of Naive Bayes, especially in high-dimensional or sparse datasets, where many conditionally independent features have very low probabilities for certain classes.
- The Naive Bayes classifier is a probabilistic model based on Bayes' Theorem and relies on the assumption that features are independent. It determines the likelihood that a data point belongs to a specific class by multiplying the class's prior probability with the conditional probabilities of each feature given that class.
- However, when these probabilities are very small, their product can suffer from floating-point underflow, causing the computed probability for some classes to become zero. This numerical issue can result in incorrect class assignments and a decline in overall model performance.

Strategies to Address Numerical Underflow

- Laplace Smoothing
- Often called **additive smoothing**, this technique is used to prevent zero probabilities in Naive Bayes, which can cause numerical issues. With this approach, a small constant (commonly 1) is added to each probability estimate. Without smoothing, if a feature never appears with a particular class, its likelihood becomes zero, causing the entire probability product for that class to be zero.
- Additive smoothing is especially helpful for managing unseen features in the dataset that would otherwise receive a probability of zero. By ensuring that no probability is exactly zero, it helps reduce the risk of numerical underflow and improves the robustness of the model.

Logarithmic Transformations

- A common and effective way to prevent numerical underflow in Naive Bayes classification is to use **logarithmic transformation**. Instead of multiplying probabilities directly—which can produce extremely small values—the logarithms of the probabilities are computed and summed. This converts the product of probabilities into a sum of log probabilities, greatly improving numerical stability.
- By working with log probabilities, the model avoids multiplying very small numbers that could otherwise lead to underflow. For this reason, most Naive Bayes implementations, such as **MultinomialNB** in scikit-learn, use log probabilities by default.